eventsProject

Gestion des rôles Participant et chaîne DevOps vers AWS EKS.

52
tests réussis

#15
Jenkins SUCCESS

72,3 %
couverture des lignes

0
bug Sonar

Release vérifiée • Pipeline unique vers EKS • Réactivation du Learner Lab requise

État vérifié le 31 octobre 2026 • Source testée : 8a5a96f

Un périmètre métier précis

Tache décrit le rôle d'un participant dans un événement. Le modèle partagé du projet est conservé.

Un enum

INVITE, ORGANISATEUR, SERVEUR et ANIMATEUR.

Un participant

Lire, affecter, remplacer ou retirer son rôle sans supprimer le participant.

Un module intégré

Contrôleurs, services et repositories existants ; aucune entité de tâche indépendante.

entities/Tache.java • services/TacheService.java

Du code au déploiement EKS

Jenkins, SonarQube et Nexus restent locaux ; la destination Kubernetes du pipeline unique est AWS EKS.

1 · GitHub / Tache

Source versionnée et checkout du commit.

2 · Jenkins

Compilation, tests et rapports JaCoCo.

3 · SonarQube

Analyse et quality gate bloquant.

4 · Terraform AWS

Plan puis apply optionnel après revue.

5 · Nexus / Docker Hub

JAR et image publiés et vérifiés.

6 · AWS EKS

Rollout et HTTP après réactivation du lab.

Jenkinsfile • Étapes AWS désactivées dans le build #15

Persister un rôle, conserver le modèle

Le rôle reste une valeur de Participant, enregistrée en chaîne dans MySQL.

```
public enum Tache {
    INVITE, ORGANISATEUR,
    SERVEUR, ANIMATEUR
}

// Participant.java
@Enumerated(EnumType.STRING)
Tache tache;
```

Lisibilité

La base contient le nom du rôle, pas sa position dans l'enum.

Intégrité

Le participant existe avant toute affectation ; les valeurs inconnues sont rejetées.

entities/Tache.java • entities/Participant.java

Un cycle de rôle vérifié par HTTP

Le JAR et l'image finale ont été testés contre le MySQL dédié, en complément des tests automatisés.

GET /taches

Consulter les quatre rôles possibles.

POST /event/addPart

Créer un participant avec un ID généré.

PUT /taches/participants/{id}

Affecter ou remplacer le rôle.

GET /taches/participants/{id}

Relire le rôle réellement persisté.

DELETE /taches/participants/{id}

Retirer le rôle ; réponse HTTP 204.

Le participant reste

Le retrait du rôle ne supprime pas sa fiche.

Limiter les champs reçus par l'API

Les DTO de requête définissent les champs modifiables. Les relations entrantes acceptent uniquement des IDs positifs.

```
@Getter
@Setter
public class RoleRequest {
    private Tache tache;
}

// Le participant vient du chemin HTTP :
service.assignRole(id, request.getTache());
```

Créations protégées

ParticipantCreateRequest, EventCreateRequest et LogisticsCreateRequest.

Champs réservés

Un client ne peut imposer l'ID généré ni le coût calculé.

controllers/dto/RoleRequest.java • EventRequests.java

Des réponses sans graphe JPA récursif

Les DTO de réponse conservent les champs scalaires ; leurs relations exposent des identifiants.

```
{
  "idPart": 7,
  "nom": "Demo",
  "tache": "SERVEUR",
  "events": [{ "idEvent": 10 }]
}

// Rôle : participantId + tache
```

Sérialisation bornée

Évite Participant → Event → Participant dans les réponses JSON.

Contrat explicite

URLs et champs scalaires conservés ; détail imbriqué remplacé par des références ID.

Exemple de JSON conforme au DTO • EventResponses.java • docs/DTO_API.md

Rejeter les erreurs avant la persistance

Le NPE historique sur une référence absente est corrigé ; les erreurs attendues sont contrôlées.

HTTP 400

Identifiant nul ou négatif, enum inconnu, rôle manquant ou référence invalide.

HTTP 404

Participant absent ou événement introuvable pour l'affectation logistique.

HTTP 204

Retrait du rôle réussi, sans suppression du participant.

EventReferenceValidationTest • TacheRestControllerTest • HTTP/MySQL réel

VÉRIFIÉ AU 31/10/2026

52 tests automatisés réussis

Jenkins #15 : zéro échec, zéro erreur et aucun test ignoré.

Service / Mockito

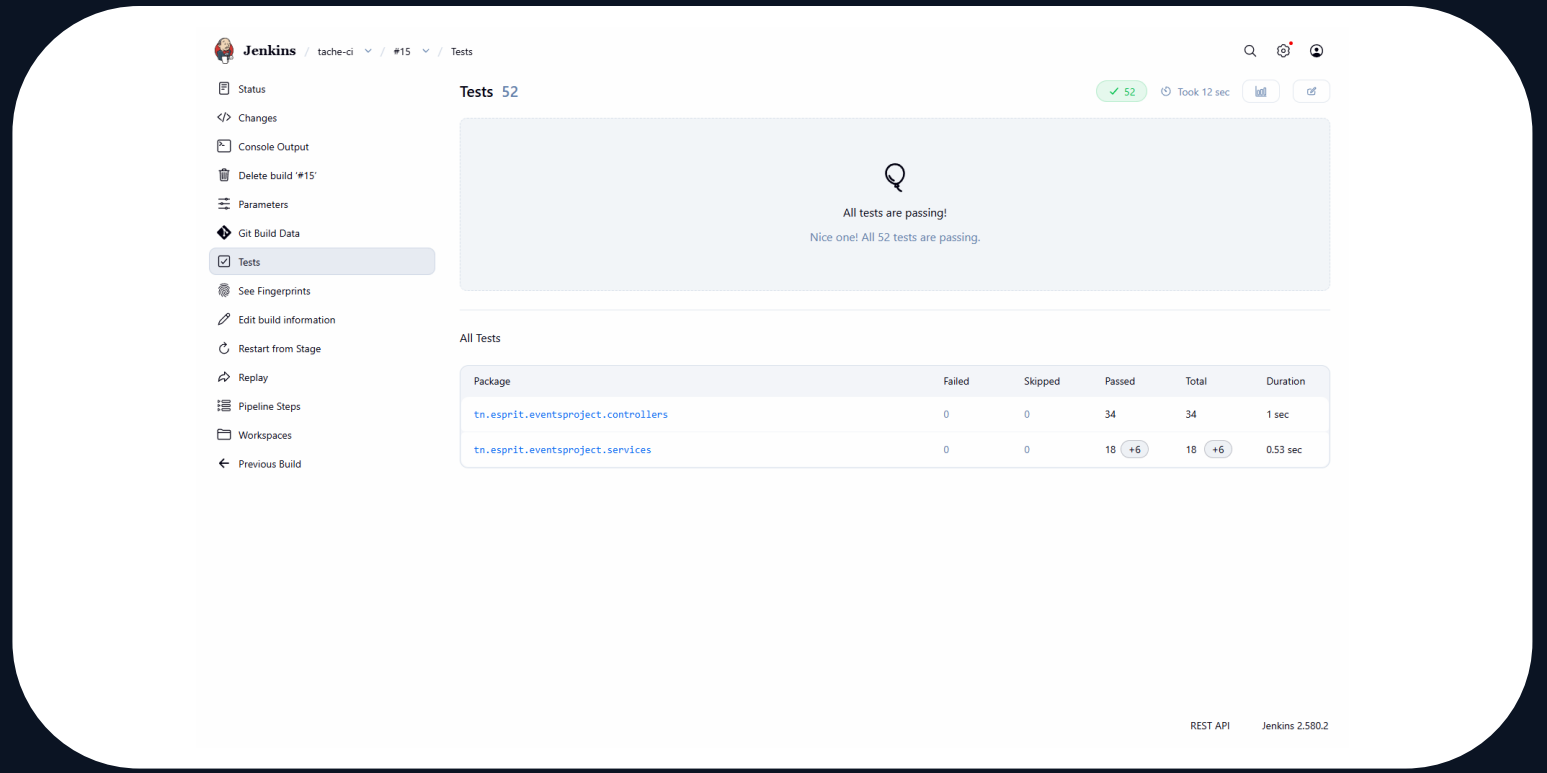
Règles de rôle et références manquantes.

HTTP / DTO

Contrats, allowlists et sérialisation cyclique.

JPA / H2

Persistance réelle ; MySQL vérifié séparément.



Jenkins #15 • Rapport JUnit authentique

VÉRIFIÉ AU 31/10/2026

JaCoCo : une couverture mesurée

Les rapports HTML et XML sont produits par Maven, archivés dans Jenkins et consommés par SonarQube.

72,3 %

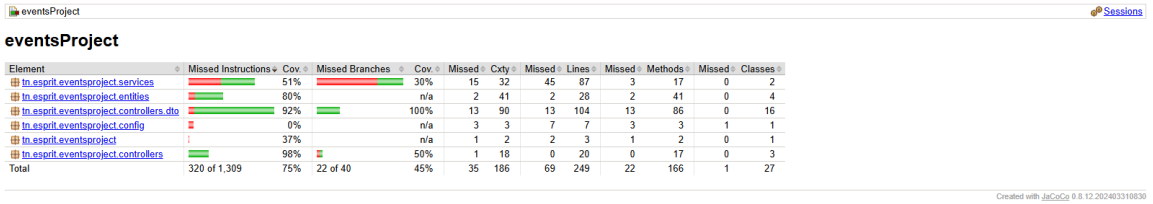
Couverture globale des lignes.

45,0 %

Couverture globale des branches.

Rapports obligatoires

Le build vérifie la présence des fichiers HTML et XML.



eventsProject

Sessions

eventsProject

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
tn.esprit.eventsproject.services		51%		30%	15	32	45	87	3	17	0	2
tn.esprit.eventsproject.entities		80%		n/a	2	41	2	28	2	41	0	4
tn.esprit.eventsproject.controllers.dto		92%		100%	13	90	13	104	13	86	0	16
tn.esprit.eventsproject.config		0%		n/a	3	3	7	7	3	3	1	1
tn.esprit.eventsproject		37%		n/a	1	2	2	3	1	2	0	1
tn.esprit.eventsproject.controllers		98%		50%	1	18	0	20	0	17	0	3
Total	320 of 1,309	75%	22 of 40	45%	35	186	69	249	22	166	1	27

Created with JaCoCo 0.8.12.202403310830

pom.xml • Jenkinsfile • 16-readiness/sonar-build-15.json

Un quality gate réellement évalué

SonarQube #15 : gate OK, conditions évaluées, aucune violation nouvelle.

0

Bug, vulnérabilité, code smell et security hotspot.

87,9 %

Couverture du nouveau code, seuil du gate : 80 %.

Périmètres distincts

Couverture globale Sonar : 68,5 % ; couverture des lignes : 72,3 %.

16-readiness/sonar-build-15.json

Tracer les changements sur Tache

Branche dédiée, commits réels et historique préservé. Les modifications de documentation sont exclues du polling.

```
fee07ba  DTO et préparation du pipeline EKS
8a5a96f  Validation des références absentes
7b7f585  Preuves et blocage du Learner Lab

Source testée par Jenkins #15 :
8a5a96f28c1e9d1fd2aa14146af43c6efcfd7bfe
```

Reproductibilité

Le tag d'image reprend le commit et le numéro de build.

Traçabilité

Chaque résultat possède un journal, un rapport ou une capture authentique.

VÉRIFIÉ AU 31/10/2026

Un seul pipeline, une release actuelle

tache-ci #15 SUCCESS. L'ancien job de promotion est désactivé ; son historique est conservé.

Compiler et tester

52 tests et rapports JaCoCo obligatoires.

Analyser

SonarQube et gate OK avant publication.

Préparer AWS

Terraform : validation, plan et approbation.

Publier

Nexus et Docker Hub réellement exécutés.

Déployer EKS

Image du build courant, plus de dépendance au build #9.

Contrôler

Rollout et HTTP ; étapes AWS ignorées dans #15.

08-jenkins/build-15-summary.json • Jenkinsfile

Un plan avant toute création AWS

Le module terraform/aws-eks partage le state AWS canonique ; le state local de supervision reste séparé.

```
RUN_TERRAFORM = true
    fmt → init → validate → plan

APPLY_TERRAFORM = false // par défaut
    revue du plan et du coût
    approbation manuelle dans Jenkins
    apply du plan sauvegardé exact
```

Garde-fous

Compte attendu, profil lab dédié et accès API limité aux IPv4 /32.

Statut actuel

Hash provider Linux corrigé ; nouveau plan requis après réactivation du lab.

Le même JAR dans toute la chaîne

Le SHA-256 du JAR Jenkins #15 est identique à celui téléchargé de Nexus et à celui présent dans l'image Docker.

Jenkins

JAR archivé après le build réussi.

Nexus

Snapshot 1.0.0-20261031.023455-10.

Docker

/app/app.jar vérifié dans le conteneur de test.

16-readiness/release-15-verification.json

Une image publique vérifiée

Le manifeste distant est accessible sans compte et identique au manifeste linux/amd64 local.

Dépôt

ayyoub1/events-tache

Tag livré

tache-8a5a96f28c1e-15

Intégrité

HTTP 200 anonyme ; digest SHA-256 local/distant identique.

Digest : 5c4ff54095b9ae76… • 16-readiness/release-15-verification.json

Tester l'image réellement construite

L'image #15 a été lancée temporairement contre MySQL, puis le conteneur de test a été supprimé.

Readiness UP

Application et connexion MySQL opérationnelles.

Métier et métriques

DTO, affectation du rôle, 404 attendu et export Prometheus vérifiés.

Utilisateur 10001

Exécution non-root, confirmée dans le conteneur.

16-readiness/docker-image-15-smoke.json

Des logs et des métriques exploitables

L'application expose des métriques Actuator ; Prometheus collecte et Grafana visualise.

```
/events/actuator/health/readiness
/events/actuator/prometheus

DEBUG  lecture du rôle
INFO   affectation / retrait
WARN   entrée invalide / participant absent
ERROR  incident inattendu, sans secret
```

Export vérifié

Métriques JVM et requêtes HTTP présentes dans le runtime testé.

Collecte EKS

Manifests et dashboard prêts ; vérification live après réactivation.

application.properties • TacheService.java • TacheExceptionHandler.java

Une infrastructure de démonstration maîtrisée

La configuration AWS vise us-east-1 et réutilise le VPC, les subnets et LabRole du laboratoire.

Quatre ressources

Cluster EKS, launch template, node group et add-on EBS CSI.

Un worker

t3.medium fixe ; disque système gp3 chiffré de 20 Gio.

Stockage applicatif

MySQL 4 Gio, Prometheus 1 Gio, Grafana 1 Gio ; accès par tunnels authentifiés.

terraform/aws-eks/ • k8s/eks/ • Création actuelle en attente de revue

EKS : démonstration réalisée le 29 octobre

Le déploiement précédent a été vérifié sur AWS puis supprimé après accord ; cette capture ne décrit pas un cluster actif aujourd'hui.

1 worker Ready

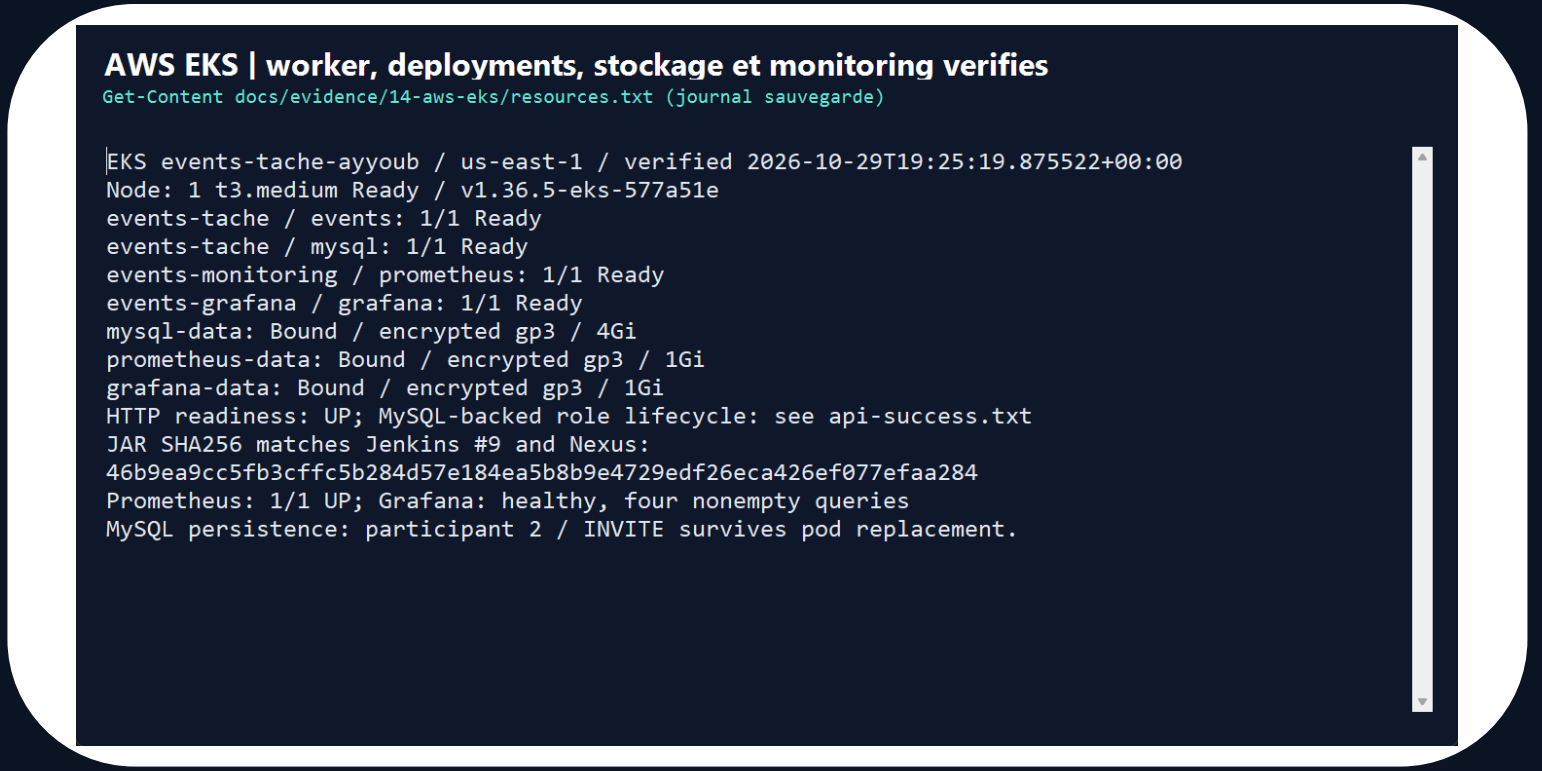
Un nœud t3.medium disponible.

4 deployments

Application, MySQL, Prometheus et Grafana.

3 PVC Bound

Volumes gp3 chiffrés vérifiés.



Capture authentique du 29/10/2026 • Infrastructure ensuite nettoyée

Prometheus : une cible réellement UP

Lors de la démonstration AWS du 29 octobre, Prometheus collectait les métriques Actuator du Service applicatif.

Collecteur

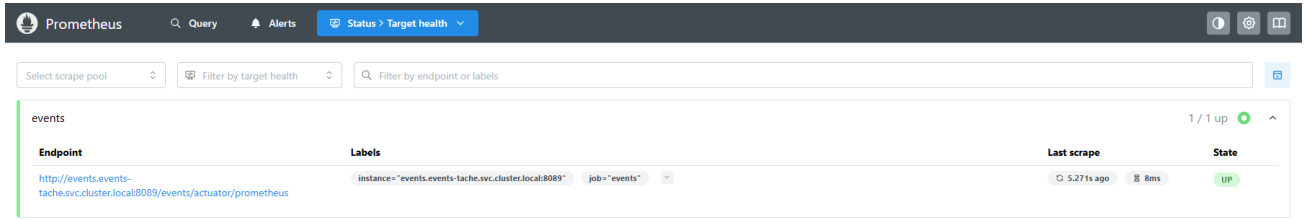
Scrape toutes les 15 secondes.

Cible applicative

Service events, port 8089, chemin /events/actuator/prometheus.

Reprise

La cible devra redevenir UP dans le nouveau cluster.



Capture du 29/10/2026 • k8s/eks/prometheus.yaml

Grafana : un dashboard provisionné

La démonstration AWS précédente valide la datasource Prometheus et quatre requêtes non vides.

Quatre vues

Cible UP, CPU Java, mémoire JVM et débit HTTP.

Provisioning

Datasource et dashboard fournis par la configuration du dépôt.

Reprise

Santé de la datasource et valeurs à vérifier après bootstrap.



Capture du 29/10/2026 • monitoring/grafana-dashboard.json

Nettoyer après la démonstration

Le nettoyage précédent a été approuvé et contrôlé : ressources Terraform et volumes de données supprimés.

Données d'abord

Suppression des PVC avant celle du cluster.

Plan de destruction

Revue puis accord avant l'apply destructif.

État confirmé

Cluster précédent absent et volumes EBS supprimés.

AWS EKS | nettoyage approuve et absence des ressources verifiee

`Get-Content docs/evidence/14-aws-eks/cleanup-verification.txt (journal sauvegarde)`

```
AWS cleanup verified: 2026-10-29T19:51:17.596155+00:00
EKS events-tache-ayyoub: absent
EC2 worker: terminated; public IPv4 released
EBS: all 3 data volumes and the root volume absent
Launch template and demo network interfaces: absent
Worker auto scaling group and EKS security group: absent
Terraform managed resources remaining: 0
Jenkins EKS job disabled; temporary credential removed; build history retained.
Local kind, source code, screenshots and presentation preserved.
Historical usage can still appear on the delayed lab bill.
```

Preuve historique • 14-aws-eks/cleanup-verification.json

Les étapes pour relancer la démonstration

La policy voc-cancel-cred refuse actuellement EKS ; le laboratoire doit être redémarré avant de préparer un nouveau plan.

1 · Démarrer le lab

Renouveler les trois credentials du profil dédié.

2 · Préparer Jenkins

Vérifier identité, compte, budget et CIDRs /32 ; renouveler les Secret files.

3 · Revoir le plan

DEPLOY_EKS=false ; approbation explicite avant apply.

4 · Bootstrap

Namespaces, Secrets, MySQL, monitoring et RBAC.

5 · Livrer

Renouveler le token EKS ; lancer le pipeline avec DEPLOY_EKS=true.

6 · Prouver

HTTP/MySQL, PVC, cible UP, datasource et dashboard.

docs/AWS_EKS_OPERATIONS.md • docs/JENKINS_TERRAFORM.md

VÉRIFIÉ AU 31/10/2026

Des résultats vérifiés, une reprise documentée

La release applicative et ses preuves sont prêtes ; la nouvelle activation AWS dépend du Learner Lab.

Application

DTO entrée/sortie, 52 tests, HTTP/MySQL et erreurs contrôlées.

Chaîne CI

Jenkins #15, gate OK, JaCoCo, Nexus et Docker Hub vérifiés.

Infrastructure

Preuves AWS historiques conservées ; plan et bootstrap à reprendre après activation du lab.

docs/READINESS.md • docs/EVALUATION_TRACKER.md

Trois choix à expliquer

Les réponses sont visibles ici ; aucune note orateur n'est ajoutée au fichier PowerPoint.

Pourquoi un DTO ?

Définir le contrat HTTP, limiter les champs modifiables et empêcher les cycles JPA.

Pourquoi un enum ?

Tache représente quatre rôles du participant, pas une tâche métier autonome.

Pourquoi plan puis apply ?

Examiner les ressources et le coût avant d'autoriser une modification AWS.